Guia de Estudo — Pokémon TCG

Versão 3 · Etapas 4, 5 e 6 (defesa final)

Projeto de Programação Orientada a Objetos · Java 17 + Swing

Repositório: `github.com/Lucasesteves-cyber/Pokmon-TCG`

Grupo: Renato e Lucas

Atualizado em 11/11/2026

Como usar este guia

> Este guia cobre **o projeto inteiro como ele está agora**, depois das Etapas 4 e 5. As partes marcadas com **NOVO** foram criadas ou mudaram nessas etapas — são as que a banca mais tende a perguntar, porque são as entregas mais recentes.
>
> Ordem sugerida: leia as seções 1 a 4 (visão geral e arquitetura), depois estude com calma as seções 5 a 10 (POO, padrões, exceções, coleções, arquivos, testes). A seção 11 é uma **referência** de todas as classes e métodos para consultar. Termine com o roteiro da apresentação (12) e as perguntas da banca (13), treinando as respostas em voz alta.

Conteúdo

1. O que mudou nas Etapas 4 e 5
2. Visão geral e como rodar
3. Arquitetura: os 8 pacotes
4. Fluxo de uma partida e de uma jogada
5. Conceitos de POO no projeto
6. Padrões de projeto (Strategy e Observer)
7. Tratamento de exceções
8. Coleções
9. Leitura e escrita de arquivos
10. Testes unitários com JUnit 5
11. Referência: todas as classes e métodos
12. Roteiro da apresentação (10–15 min)
13. Perguntas prováveis da banca (com respostas)
14. Checklist do dia da defesa

Os números do projeto

Item	Quantidade
Pacotes de código	8 (`modelo`, `efeitos`, `eventos`, `excecoes`, `fabrica`, `persistencia`, `servicos`, `ui`)
Classes, interfaces, enums e records	34 no código + 7 classes de teste
Testes JUnit	55, todos passando
Padrões GoF	2 — Strategy e Observer (mais Simple Factory e Null Object como apoio)
Exceções próprias	5
Bibliotecas externas no jogo	Nenhuma (só o JUnit, para os testes)

1. O que mudou nas Etapas 4 e 5

Resumo do "antes e depois". Se a banca perguntar "o que vocês fizeram nessas etapas?", esta tabela é a resposta.

Assunto	Antes	Depois
Erros de regra	`Jogador` fazia `System.out.println("erro")` e devolvia `false`. A interface não sabia o motivo do erro.	O `Jogador` lança `JogadaInvalidaException` com a mensagem. A interface mostra a mensagem num único método (`executarJogada`).
Baralho vazio	`comprarCarta()` devolvia `false`.	Lança `BaralhoVazioException`; a interface encerra a partida com derrota.
Falhas de internet	`catch (Exception e) { return null; }` espalhados no `AppGUI`.	Classe `PokeApiCliente` lança `PokeApiException` (verificada) com mensagem clara.
Mensagens na tela	O `AppGUI` redirecionava o `System.out` para a tela com `System.setOut`. Isso quebrava acentos/emojis.	**Observer**: o `Jogador` avisa os ouvintes; a tela, o console e os testes recebem os eventos.
Arquivos	Nenhum.	Histórico de partidas em `dados/historico_partidas.csv` (escrita + leitura para o placar).
Testes	Nenhum.	55 testes JUnit 5 em `teste/`.
Times fixos	Ficavam no `App` (console) e o `AppGUI` chamava `App.adicionarTimeAgua`.	Enum `TimeFixo` + `BaralhosPadrao` no pacote `fabrica`; times disponíveis num `EnumSet`.
Fraquezas de tipo	Cadeia de `if` dentro do `atacar`.	Classe `TabelaDeTipos` com um `Map`.
Duplicação	Dois códigos iguais para escolher evolução, dois para checar vitória, `iniciar` e `reiniciarTrocandoEquipe` quase idênticos, métodos `usarPocao/usarTroca...` repetindo a validação.	Extraídos para `escolherEvolucaoNaMao`, `verificarVitoriaAposAtaque`, `montarComposicao`, `cartaDaMao`; atalhos removidos.
"Factory Method"	O README chamava a `CartaPokemonFactory` de Factory Method (GoF).	Corrigido: é uma **Simple Factory** (não-GoF). Os 2 GoF são Strategy e Observer.
Javadoc	Parcial.	Todas as classes e métodos públicos.

Bugs encontrados e corrigidos no caminho

- **Poção curava 40**, mas o texto da carta e o README diziam 30. Agora cura 30 (constante `EfeitoPocao.CURA`). Um teste garante isso.
- **Acentos e emojis embaralhados** no histórico da tela: o `PrintStream` falso convertia cada *byte* do UTF-8 num caractere. O Observer eliminou o problema.
- **Terminal fechava** se o usuário digitasse uma letra onde se esperava número (`InputMismatchException`). Agora pergunta de novo.
- **Recuo com posição inválida**: as validações agora acontecem antes de pagar o custo de energia (um teste cobre isso).
- **Notificações longas** eram cortadas na borda da tela; agora quebram linha.

2. Visão geral e como rodar

O projeto é um **jogo de cartas colecionáveis (TCG) inspirado no Pokémon TCG**, em Java puro com interface gráfica Swing. Dois modos: **PvP** (dois jogadores no mesmo PC) e **Solo** (contra um Bot). Cada jogador monta um time (fixo: Água, Fogo, Planta; ou customizado com Pokémon da 1ª à 6ª geração, buscados na PokeAPI), coloca Pokémon em campo, anexa energia, evolui, usa cartas de Treinador e ataca. Vence quem conquistar os 2 prêmios, deixar o adversário sem Pokémon em campo, ou se o adversário ficar sem cartas para comprar.



Tabuleiro no Turno 1. No canto, a notificação gerada por uma `JogadaInvalidaException` (tentativa de atacar no primeiro turno).

Regras principais (e onde estão no código)

Regra	Onde
Mão inicial de 6 cartas; quem começa é sorteado no Cara ou Coroa	`AppGUI.iniciarPartida`, `SorteioMoeda`
Ninguém compra carta no próprio 1º turno; depois compra 1 por turno	`AppGUI.iniciarTurno`
Só Pokémon Básico entra direto em campo; Banco com no máximo 5	`Jogador.colocarPokemonEmCampo`
1 energia por rodada; cada carta aceita de 6 a 8 energias (sorteado)	`Jogador.anexarEnergia`, `CartaPokemon.anexarEnergia`
Ataque e evolução só a partir do Turno 2; 1 ataque por rodada; atacar encerra o turno	`Jogador.atacar`, `Jogador.evoluir`, `AppGUI.acaoAtacar`
Água > Fogo > Planta > Água: dano dobrado	`TabelaDeTipos.eSuperEfetivo`
Evoluir mantém energias e o dano já sofrido	`Jogador.executarEvolucao`
Recuar custa 1 energia e só 1 vez por turno; a carta Troca recua de graça	`Jogador.recuar`, `EfeitoTroca`
Nocaute: vai para a Zona Morta, atacante ganha 1 prêmio, o 1º do Banco sobe	`Jogador.resolverNocaute`

Como compilar e rodar

Na pasta `Projeto - Teste`, o jeito mais fácil é dar dois cliques em `rodar.bat` (jogo) ou `rodar-testes.bat` (testes). Pelo terminal:

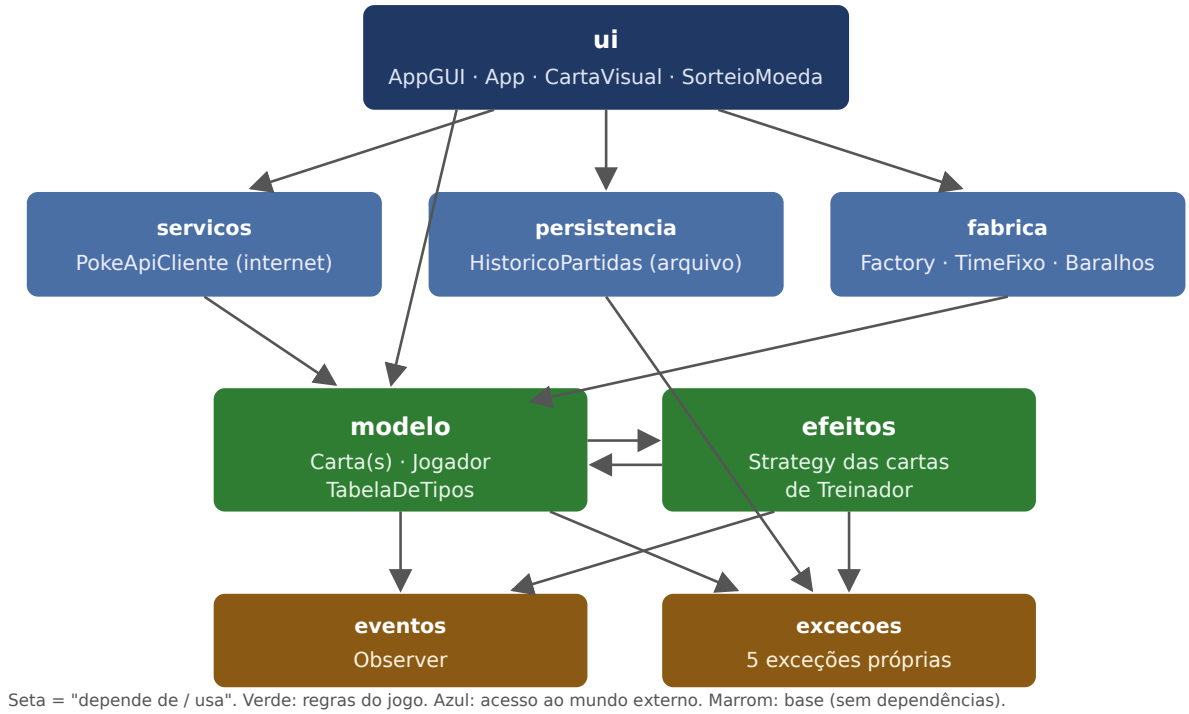
```
# compilar o jogo (todos os pacotes) e abrir a versão gráfica
javac -encoding UTF-8 -d bin modelo/*.java efeitos/*.java eventos/*.java excecoes/*.java fabrica/*.java
persistencia/*.java servicos/*.java ui/*.java
java -cp bin ui.AppGUI

# versão em terminal (protótipo)
java -cp bin ui.App
```

- `-encoding UTF-8`: os fontes têm acentos; sem isso o Windows pode ler errado.
- `-d bin`: os `.class` vão para a pasta `bin` (que não vai para o GitHub, está no `.gitignore`).
- `-cp bin`: diz ao `java` onde procurar as classes. `ui.AppGUI` é o nome completo (pacote + classe) da classe que tem o `main`.

3. Arquitetura: os 8 pacotes

O código é separado **por responsabilidade**. A regra geral: pacotes de cima conhecem os de baixo, nunca o contrário. As regras do jogo (`modelo`) não sabem que existe tela, internet ou arquivo.



Seta = "depende de / usa". Verde: regras do jogo. Azul: acesso ao mundo externo. Marrom: base (sem dependências).

Dependências entre os pacotes.

Pacote	Responsabilidade	Classes
`modelo`	Regras e estado do jogo. Não conhece Swing, arquivo nem internet.	`Carta` (abstrata), `CartaPokemon`, `CartaEnergia`, `CartaTreinador`, `Jogador`, `TabelaDeTipos` **NOVO**
`efeitos`	Padrão Strategy: o comportamento de cada carta de Treinador.	`EfeitoTreinador` (interface), `EfeitoPocao`, `EfeitoCuraTotal`, `EfeitoTroca`, `EfeitoEvolucaoRapida`, `EfeitoNenhum`, `EfeitoTreinadorRegistro`, `TipoAlvo` (enum), `ParametrosEfeito`
`eventos` **NOVO**	Padrão Observer: avisar quem estiver interessado no que acontece na partida.	`OuvinteJogo` (interface), `EventoJogo` (record), `TipoEvento` (enum), `OuvinteConsole`
`excecoes` **NOVO**	Erros específicos do sistema, com mensagens úteis.	`JogoException`, `JogadaInvalidaException`, `BaralhoVazioException`, `PokeApiException`, `PersistenciaException`
`fabrica`	Criação de cartas e montagem dos baralhos padrão.	`CartaPokemonFactory`, `BaralhosPadrao` **NOVO**, `TimeFixo` (enum) **NOVO**
`persistencia` **NOVO**	Ler e gravar o histórico de partidas em arquivo.	`HistoricoPartidas`, `ResultadoPartida` (record)
`servicos` **NOVO**	Acesso à PokeAPI (internet).	`PokeApiCliente`
`ui`	Interfaces com o usuário: gráfica e terminal.	`AppGUI`, `App`, `CartaVisual`, `SorteioMoeda`

> **Ponto que a banca pode notar:** `modelo` e `efeitos` dependem um do outro (o `Jogador` chama a estratégia, e a estratégia recebe o `Jogador`). É uma dependência circular *intencional*: no Strategy, o contexto (Jogador) passa a si mesmo para a estratégia agir sobre ele. Os dois pacotes formam juntos o "núcleo" do jogo. Se perguntarem, essa é a explicação.

Por que separar assim?

- **Coesão:** cada pacote tem um assunto só. Fica fácil achar onde mexer.
- **Baixo acoplamento:** dá para trocar a interface (terminal ↔ gráfica) sem mexer nas regras. Prova disso: o `App` (terminal) e o `AppGUI` usam exatamente o mesmo `Jogador`.
- **Testabilidade:** como o `modelo` não depende de tela, os testes JUnit criam um `Jogador` e testam as regras sem abrir janela nenhuma.

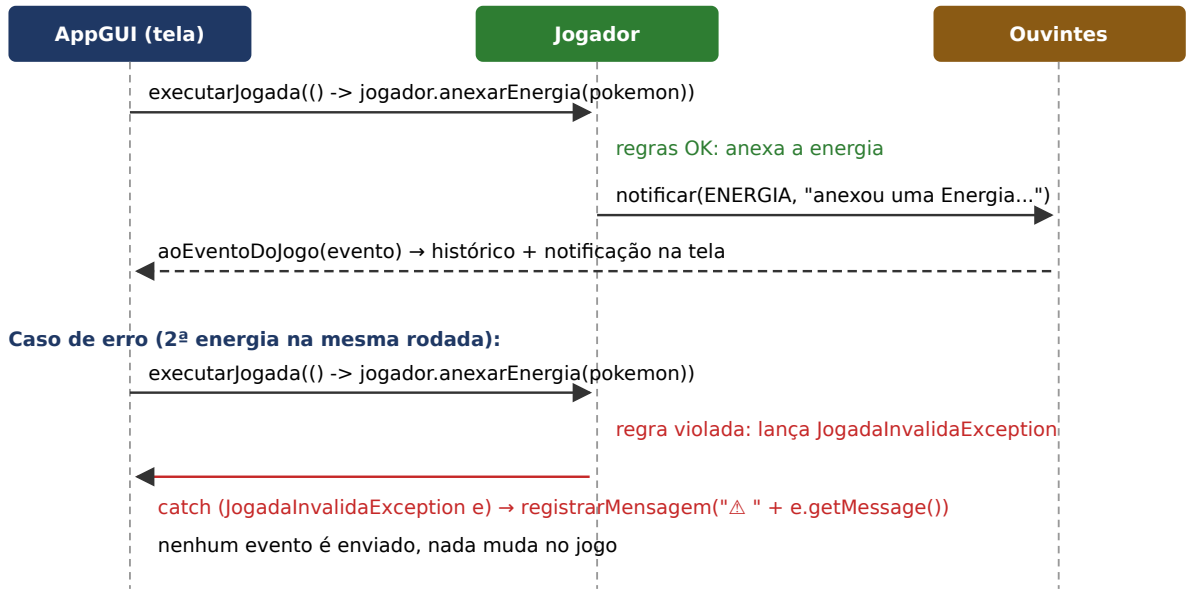
4. Fluxo de uma partida e de uma jogada

4.1 Do `main` até o fim da partida

1. `AppGUI.main` aplica o tema do Swing e chama `iniciar()` na thread do Swing.
2. `iniciar()`: pergunta o modo (`escolherModoDialog`), o nome do Treinador 1 e chama `montarComposicao` — que abre `escolherTimeDialog` (time fixo ou customizado). Depois faz o mesmo para o Treinador 2, ou, no modo Solo, sorteia o time do Bot na PokeAPI (`construirTimeAleatorioBotComLoading`).
3. `iniciarPartida(...)`: cria os dois `Jogador`es, adiciona cartas de treinador e prêmios (`BaralhosPadrao`), embaralha, compra 6 cartas cada, sorteia quem começa (`SorteioMoeda`), monta a janela e **inscreve a tela como ouvinte** dos dois jogadores (`adicionarOuvinte(this::aoEventoDoJogo)`).
4. `iniciarTurno()`: o jogador da vez compra 1 carta (exceto no 1º turno de cada um) — se o baralho acabou, `BaralhoVazioException` → `encerrarJogoComDerrota`. Libera as evoluções e redesenha a tela. Se for a vez do Bot, agenda `executarTurnoBot`.
5. O humano clica nas cartas e botões (`acaoClicarCartaPokemon`, `abrirMenuPokemon`, `acaoClicarCartaTreinador`, `acaoAtacar`, `acaoPassarTurno`). Cada jogada passa por `executarJogada`.
6. Depois de um ataque, `verificarVitoriaAposAtaque` confere se alguém venceu. Se sim: `mostrarTelaFimDeJogo`, que **grava o resultado no arquivo** e mostra o placar geral. Se não: `encerrarRodada` + `trocarTurno` → volta ao passo 4.

4.2 O caminho de UMA jogada (o mais importante para explicar)

Exemplo: o jogador clica num Pokémon e escolhe "Anexar Energia". Repare como exceções e Observer trabalham juntos:



Sucesso: o Jogador avisa os ouvintes (Observer). Erro: o Jogador lança exceção, e o `executarJogada` mostra a mensagem.

```
// AppGUI — o ÚNICO lugar que trata jogada inválida do humano
private boolean executarJogada(Runnable jogada) {
    try {
        jogada.run();          // ex.: jogadorAtual.anexarEnergia(pokemon)
        return true;
    } catch (JogadaInvalidaException e) {
        registrarMensagem("⚠ " + e.getMessage());   // mostra o motivo na tela
        return false;
    }
}
```

`Runnable` é uma interface funcional do Java (um método `run()` sem parâmetros). A expressão `() -> jogadorAtual.anexarEnergia(pokemon)` é uma **lambda**: um "pedaço de código" passado como parâmetro. Assim o mesmo `executarJogada` serve para qualquer ação.

4.3 O turno do Bot

`executarTurnoBot()` é uma IA simples, em ordem: (1) se não tem Ativo, coloca o primeiro Básico da mão como Ativo; (2) enche o Banco com os Básicos restantes; (3) evolui o Ativo se tiver a evolução na mão; (4) anexa energia no Ativo; (5) ataca se tiver energia. Cada ação passa por `tentarJogadaDoBot`, que faz o mesmo que o `executarJogada`, mas **não** mostra notificação: se a jogada for inválida (ex.: evoluir no Turno 1), só registra no histórico e segue para a próxima ação.

5. Conceitos de POO no projeto

Para cada conceito: a definição curta (para falar na banca) e **onde mostrar no código**.

Conceito	Definição curta	Onde está no projeto
Classe e objeto	Classe é o molde; objeto é uma instância criada com `new`.	`new Jogador("Ash")`, `new CartaPokemon("Squirtle", "Água", 50, 15)`.
Encapsulamento	Atributos privados; o acesso é controlado por métodos que validam as regras.	O HP da `CartaPokemon` é `private`; só muda por `receberDano` (não deixa negativo) e `curar` (não passa do máximo). O `Jogador` só aceita energia por `anexarEnergia`, que confere o limite por rodada.
Herança	Uma classe filha reaproveita e especializa a classe mãe (`extends`).	`CartaPokemon`, `CartaEnergia` e `CartaTreinador` herdam de `Carta`. `JogadaInvalidaException` herda de `JogoException`, que herda de `RuntimeException`. `CartaVisual` herda de `JButton`; `SorteioMoeda` de `JDialog`.
Classe abstrata	Não pode ser instanciada; serve de base comum.	`Carta` é `abstract`: não existe "uma carta qualquer", só Pokémon, Energia ou Treinador.
Interface	Contrato: diz *o que* fazer, sem dizer *como*.	`EfeitoTreinador` (Strategy) e `OuvinteJogo` (Observer). `OuvinteJogo` é `@FunctionalInterface`: dá para implementar com lambda.
Polimorfismo	A mesma chamada se comporta diferente conforme o objeto real.	`efeito.aplicar(...)` cura, troca ou evolui dependendo da estratégia. `ouvinte.aoOcorrerEvento(...)` mostra na tela, imprime ou guarda numa lista. `toString()` sobrescrito em cada carta. `paintComponent` sobrescrito na `CartaVisual`.
Sobrecarga	Mesmo nome de método, parâmetros diferentes.	Dois construtores em `CartaPokemon` (com e sem `evoluiDe`); dois `criarFixo` na fábrica; três construtores na `CartaVisual`; dois na `PokeApiException`.
Composição / agregação	Um objeto "tem" outros objetos.	`Jogador` tem baralho, mão, Banco e prêmios (listas de cartas). `CartaPokemon` tem uma lista de `CartaEnergia`. `CartaTreinador` tem uma `EfeitoTreinador`.
Abstração	Esconder detalhes e mostrar só o essencial.	A tela chama `pokeApi.buscarPokemon(25)` sem saber de HTTP, JSON ou regex. Chama `historico.registrar(...)` sem saber de CSV.
Enum	Tipo com um conjunto fixo de valores.	`TipoAlvo`, `TipoEvento`, `TimeFixo`. O `TimeFixo` tem atributos e comportamento (`montar`), não só nomes.
Record NOVO	Classe imutável só de dados; o Java gera construtor, getters, `equals`, `hashCode`, `toString`.	`EventoJogo(tipo, mensagem)` e `ResultadoPartida(...)`. O teste `gravaELe` usa o `equals` gerado para comparar o que foi gravado com o que foi lido.
Membros estáticos	Pertencem à classe, não ao objeto.	Constantes (`Jogador.LIMITE_BANCO`, `EfeitoPocao.CURA`), métodos da fábrica, `TabelaDeTipos.eSuperEfetivo`.
Lambda / referência de método	Função passada como valor.	`jogador.adicionarOuvinte(this::aoEventoDoJogo)`; `executarJogada(() -> ...)`; `TimeFixo` guarda `BaralhosPadrao::adicionarTimeAgua`.

Princípios de design que dá para citar

- **Responsabilidade única (SRP):** `PokeApiCliente` só fala com a API; `HistoricoPartidas` só cuida do arquivo; `TabelaDeTipos` só sabe as fraquezas. Antes tudo isso estava dentro do `AppGUI`.

- **Aberto/fechado (OCP):** nova carta de Treinador = nova classe de estratégia, sem alterar `Jogador` nem `AppGUI`. Novo jeito de mostrar eventos = novo ouvinte, sem alterar `Jogador`.
- **DRY (não se repita):** validações centralizadas (`cartaDaMao`, `trocarAtivoComBanco` usada pelo recuo e pela Troca).

6. Padrões de projeto

A Etapa 5 pede **pelo menos 2 padrões GoF**, com justificativa. O projeto usa **Strategy** e **Observer**. Os dois resolvem problemas reais que o código tinha.

6.1 Strategy — efeitos das cartas de Treinador

Ideia do padrão: uma família de algoritmos intercambiáveis, cada um numa classe, todos com a mesma interface. O objeto que usa (contexto) não sabe qual algoritmo concreto está executando.

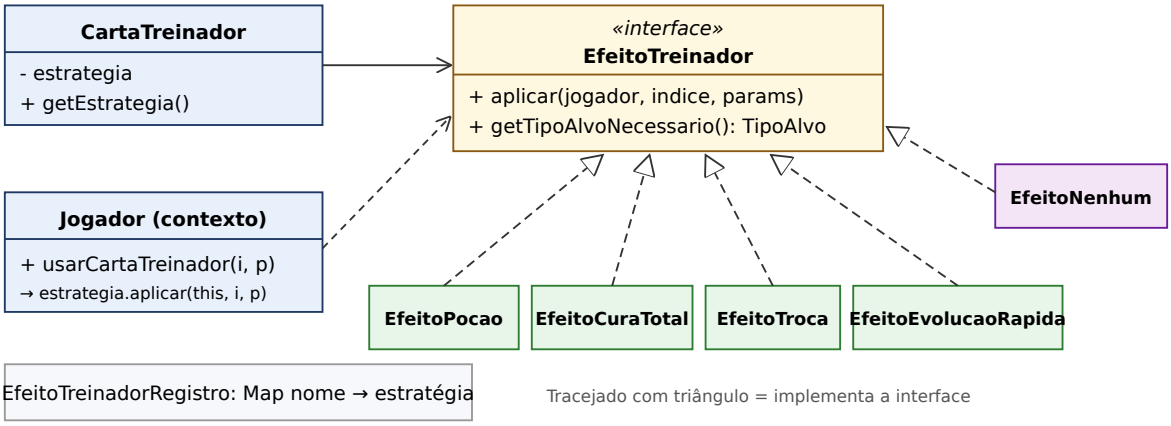


Diagrama de classes simplificado do Strategy.

Problema que resolveu: antes, a tela decidia o efeito comparando texto (`if (efeito.equals("Poção")) ... else if ("Troca") ...`) e o `Jogador` tinha um método por carta (`usarPocao`, `usarTroca`...), todos repetindo a mesma validação. Para criar uma carta nova era preciso mexer em várias classes.

Como funciona hoje:

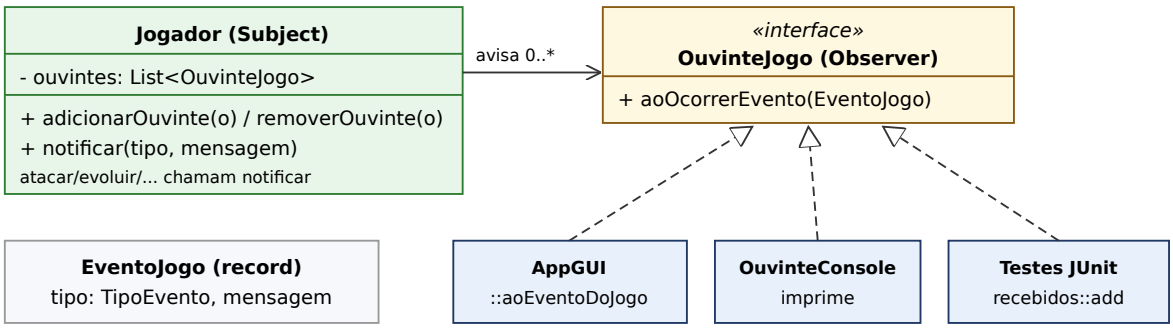
1. Ao criar `new CartaTreinador("Poção", "Poção")`, o construtor pede ao `EfeitoTreinadorRegistro.obter("Poção")` a estratégia certa (um `HashMap` nome → objeto).
2. A tela pergunta à estratégia `getTipoAlvoNecessario()` para saber qual diálogo abrir (escolher Pokémon em campo, posição do Banco, evolução + alvo, ou nada) e monta um `ParametrosEfeito`.
3. `jogador.usarCartaTreinador(indice, parametros)` valida a carta e chama `estrategia.aplicar(this, indice, parametros)`. Pronto — polimorfismo.

Por que é adequado: cada carta é uma variação do mesmo comportamento ("aplicar um efeito"), escolhida em tempo de execução. Para adicionar uma carta nova (ex.: "Super Poção"), basta criar `EfeitoSuperPocao implements EfeitoTreinador` e registrar uma linha no registro. Nada mais muda.

Bônus — Null Object: `EfeitoNenhum` é a estratégia da carta Troféu e de nomes desconhecidos. Em vez de `null` (que causaria `NullPointerException`), existe um objeto que simplesmente explica "esta carta não tem efeito".

6.2 Observer — eventos da partida NOVO

Ideia do padrão: um objeto (o *Subject*) mantém uma lista de observadores e avisa todos quando algo acontece. O Subject não sabe o que cada observador faz com o aviso. Relação um-para-muitos com baixo acoplamento.



Três ouvintes diferentes reagem ao mesmo evento — e o Jogador não conhece nenhum deles.

Diagrama de classes simplificado do Observer.

Problema que resolveu: as regras (`Jogador` e efeitos) faziam `System.out.println` para tudo. Para a mensagem aparecer na janela, o `AppGUI` trocava a saída do console por uma falsa (`System.setOut`), que transformava cada byte em letra. Consequências: (1) as regras ficavam presas ao console; (2) era impossível testar se um evento aconteceu; (3) **acentos e emojis apareciam embaralhados**, porque "ç" em UTF-8 tem 2 bytes.

```
// Jogador (Subject)
public void notificar(TipoEvento tipo, String mensagem) {
    EventoJogo evento = new EventoJogo(tipo, mensagem);
    for (OuvinteJogo ouvinte : new ArrayList<>(ouvintes)) {   // cópia: um ouvinte pode se remover no
meio
        ouvinte.aoOcorrerEvento(evento);
    }
}

// AppGUI se inscreve nos dois jogadores ao iniciar a partida
jogador1.adicionarOuvinte(this::aoEventoDoJogo);
jogador2.adicionarOuvinte(this::aoEventoDoJogo);

// App (terminal) usa outro ouvinte; o teste usa uma lista
jogador.adicionarOuvinte(new OuvinteConsole());
jogador.adicionarOuvinte(recebidos::add);
```

Por que é adequado: um jogador, vários interessados, e o jogador não deve depender de quem escuta. O `TipoEvento` permite reagir diferente por tipo: a tela usa `TipoEvento.COMPRA` para **esconder do humano qual carta o Bot comprou** ("Bot comprou uma carta"), sem nenhuma mudança no `Jogador`.

Detalhe técnico: eventos podem vir de outra thread; o `aoEventoDoJogo` confere `SwingUtilities.isEventDispatchThread()` e, se preciso, reagenda com `invokeLater` — no Swing só a thread de eventos pode mexer na tela.

6.3 E a CartaPokemonFactory?

> **Cuidado na defesa:** a `CartaPokemonFactory` é uma **Simple Factory** (classe com métodos `static` que centralizam a criação). Ela **não** é o *Factory Method* do GoF, que usa herança: uma classe criadora abstrata declara o método de criação e subclasses decidem o que criar. Por isso ela é apresentada como recurso complementar, não como um dos 2 GoF. Se a banca perguntar, explicar essa diferença mostra domínio do assunto.

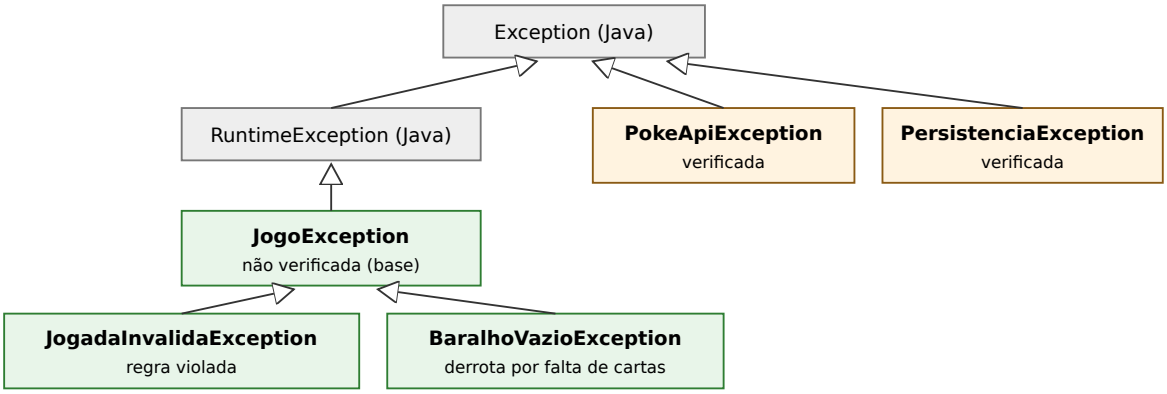
O que ela resolve: havia 4 lugares fazendo `new CartaPokemon(...)` com a mesma lógica de copiar carta ou aplicar HP/dano padrão quando a PokeAPI não informava. Agora há `criarDaPokeAPI` (aplica `HP_PADRAO`=50 e `DANO_PADRAO`=20), `criarFixo` (times prontos) e `clonar` (cópia independente com HP cheio).

7. Tratamento de exceções NOVO

7.1 Conceitos rápidos

- **Exceção**: objeto que representa um erro; é *lançada* com `throw` e *capturada* com `try/catch`. Se ninguém captura, o programa para.
- **Verificada (checked)**: herda de `Exception`. O compilador **obriga** a tratar (`catch`) ou declarar (`throws`). Usada para falhas externas previsíveis: rede, disco.
- **Não verificada (unchecked)**: herda de `RuntimeException`. Não obriga `throws`. Usada para erros de uso/lógica.
- **`finally`**: sempre executa (o `PokeApiCliente` usa para fechar a conexão). **try-with-resources**: `try (BufferedReader r = ...) { }` fecha o recurso sozinho (usado no histórico).
- **Causa**: `new PokeApiException("mensagem", e)` guarda a exceção original (`getCause()`), para não perder a informação técnica.

7.2 A hierarquia do projeto



Exceção	Quem lança e quando	Quem trata e como
`JogadaInvalidaException`	`Jogador` (todas as ações) e as estratégias de `efeitos`: atacar no Turno 1, 2ª energia, evolução errada, Banco cheio, posição inválida, sem energia...	Humano: `AppGUI.executarJogada` mostra a mensagem. Bot: `tentarJogadaDoBot` registra e segue. Terminal: `catch` no loop do `App.main`.
`BaralhoVazioException`	`Jogador.comprarCarta()` quando o baralho está vazio. Guarda o nome do jogador (`getNomeJogador()`).	`AppGUI.iniciarTurno` → `encerrarJogoComDerrota`. No terminal, anuncia o vencedor.
`PokeApiException`	`PokeApiCliente`: sem internet (`IOException` embrulhada), HTTP 404/erro, número fora de 1–721, JSON inesperado.	Montador de time: mostra "❌ Verifique sua conexão...". Sorteio do Bot: pula aquele Pokémon. Lista de tipos: segue sem o ícone.
`PersistenciaException`	`HistoricoPartidas`: erro de disco ao gravar ou ler (embrulha a `IOException` com o caminho do arquivo).	`salvarResultadoEMontarPlacar`: mostra o aviso na tela de fim de jogo; o jogo continua.
`IllegalArgumentException` (do Java)	Construtores: `Jogador` com nome vazio, `CartaPokemon` com HP ≤ 0 ou dano negativo.	Erro de programação: não deveria acontecer. No terminal, o nome é pedido de novo.

7.3 Por que umas são verificadas e outras não? (pergunta clássica)

> "Jogada inválida é erro do **jogador** e pode acontecer em qualquer ação. Se fosse verificada, todo método do Jogador teria `throws` e a tela teria `try/catch` em cada botão. Como é não verificada, tratamos tudo num lugar só, o `executarJogada`. Já rede e disco são falhas **externas** e previsíveis: aí queremos que o compilador obrigue quem chama a pensar no que fazer, por isso `PokeApiException` e `PersistenciaException` são verificadas."

7.4 Mensagens úteis (exemplos reais)

- `Charmeleon não evolui de Squirtle! Ele evolui de Charmander.` — diz o erro e a solução.
- `Wartortle é uma evolução (evolui de Squirtle)! Ele não pode entrar direto em campo — evolua um Squirtle que já esteja em campo.`
- `Posição inválida na mão! Escolha uma carta entre 1 e 6.`
- `Número da Pokédex inválido: 0. O jogo aceita do 1 ao 721.`
- `Não foi possível salvar o histórico em "D:\...\dados\historico_partidas.csv": ...`

7.5 Os `catch` que ficaram (e por quê)

Nenhum `catch (Exception e) {}` vazio sobrou. Os que existem capturam tipos específicos e têm comentário: tema do Swing indisponível (`ReflectiveOperationException | UnsupportedLookAndFeelException` — segue com o tema padrão), imagem não baixou (`IOException` — segue sem ilustração), entrada não numérica no terminal (`InputMismatchException` — pergunta de novo), linha corrompida no CSV (`IllegalArgumentException | DateTimeParseException` — pula a linha e conta).

8. Coleções

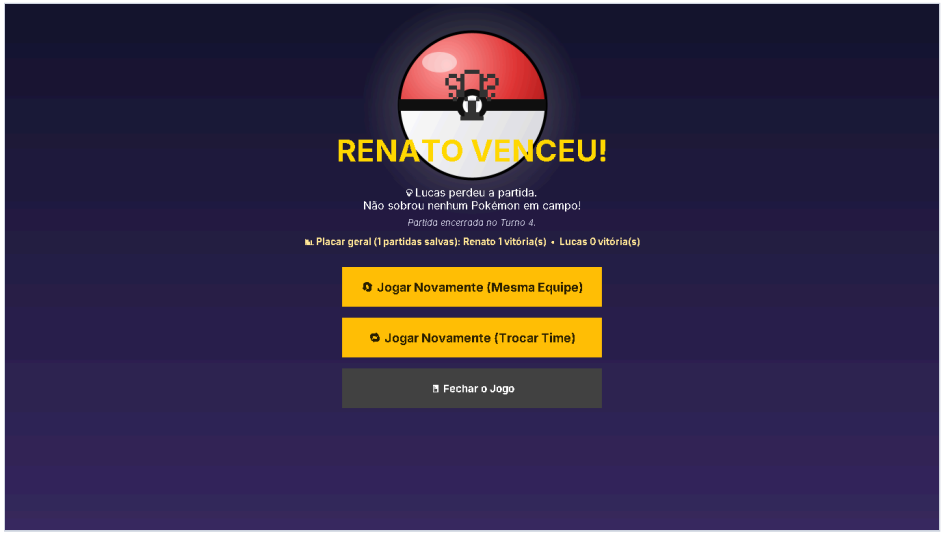
Coleção é uma estrutura que guarda vários objetos. As 3 interfaces principais: **`List`** (ordenada, aceita repetidos, acesso por índice), **`Set`** (sem repetidos), **`Map`** (pares chave → valor). A regra do projeto: declarar pela interface (`List`) e instanciar a implementação (`new ArrayList<>()`).

Coleção	Onde	Por que essa escolha
`List<Carta>` / `ArrayList`	`Jogador`: baralho, mão, prêmios	A ordem importa (topo do baralho = índice 0) e a interface escolhe cartas pela posição. Aceita cartas repetidas (4 Squirtles).
`List<CartaPokemon>`	Banco e Zona Morta	Posição no Banco importa ("Banco 1", "Banco 2"); após nocaute, sobe o índice 0.
`List<CartaEnergia>`	`CartaPokemon.energiasAnexadas`	Conta energias; o recuo remove a primeira.
`List<OuvinteJogo>` **NOVO**	Ouvintes do `Jogador`	Ordem de inscrição; não inscreve o mesmo ouvinte duas vezes (`contains`).
`Map<String, EfeitoTreinador>` / `HashMap`	`EfeitoTreinadorRegistro`	Busca a estratégia pelo nome em tempo constante, sem `if/else`.
`Map.of(...)` imutável **NOVO**	`TabelaDeTipos`, tradução de tipos (`PokeApiCliente`)	Tabelas fixas; `Map.of` não deixa ninguém alterar por engano.
`LinkedHashMap<String, Integer>` **NOVO**	`HistoricoPartidas.vitoriasPorJogador`	Conta vitórias (`merge(nome, 1, Integer::sum)`) e **mantém a ordem de inserção**, para o ranking sair do maior para o menor.
`Set<TimeFixo>` / `EnumSet` **NOVO**	Escolha de times (`AppGUI`, `App`)	Cada time só pode ser escolhido uma vez. `remove(time)` já devolve se ele estava disponível. `EnumSet` é a implementação de `Set` otimizada para enums.
`Set<Integer>` / `HashSet` **NOVO**	`PokeApiCliente.buscarLinhaEvolutiva`	Guarda os números já visitados: evita laço infinito se a API devolver uma cadeia circular.
`Map` + `Set` de cache	`AppGUI`: sprites e tipos	O `Map` guarda a imagem já baixada; o `Set` marca o que está sendo baixado agora, para não disparar o mesmo download duas vezes.
`BlockingQueue`	`AppGUI.construirTimeAleatorioBot`	Várias threads buscam Pokémon na API ao mesmo tempo e depositam o resultado numa fila segura para threads.

9. Leitura e escrita de arquivos NOVO

Classe: `persistencia/HistoricoPartidas`. Ao fim de cada partida, o resultado é **gravado** numa linha de `dados/historico_partidas.csv`; logo depois o arquivo é **lido** de volta para mostrar o placar geral.

```
data_hora;vencedor;perdedor;modo;turnos;motivo
2026-11-11T10:42:13;Renato;Bot;Solo;14;Conquistou os 2 prêmios!
2026-11-11T11:05:40;Lucas;Renato;PvP;9;Não sobrou nenhum Pokémon em campo!
```



Tela de fim de jogo com o placar lido do arquivo.

Escrita — `registrar(ResultadoPartida)`

```java
public void registrar(ResultadoPartida resultado) throws PersistenciaException {
    try {
        Files.createDirectories(pasta);                       // cria "dados/" se não existir
        boolean arquivoNovo = Files.notExists(arquivo) || Files.size(arquivo) == 0;
        try (BufferedWriter escritor = Files.newBufferedWriter(arquivo, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {   // acrescenta no fim
            if (arquivoNovo) { escritor.write(CABECALHO); escritor.newLine(); }
            escritor.write(paraLinha(resultado));
            escritor.newLine();
        }                                                     // try-with-resources fecha o arquivo
    } catch (IOException e) {
        throw new PersistenciaException("Não foi possível salvar o histórico em ...", e);
    }
}
```

Leitura — `carregar()`

- Se o arquivo não existe ainda: devolve lista vazia (não é erro).
- Lê linha a linha com `BufferedReader` em UTF-8; pula o cabeçalho e linhas em branco.
- Cada linha vira um `ResultadoPartida` (`deLinha`: `split(";")`, `LocalDateTime.parse`, `Integer.parseInt`).
- Linha corrompida (alguém editou à mão)? Captura `IllegalArgumentException | DateTimeParseException`, **pula e conta** (`getLinhasIgnoradasNaUltimaLeitura`) em vez de derrubar o jogo.

Detalhes que valem citar

- **UTF-8 explícito** na leitura e na escrita: nomes com acento funcionam em qualquer Windows.
- **Proteção do formato**: `limpar()` troca `;` e quebras de linha nos nomes, senão um nome "Ash;Ketchum" criaria uma coluna a mais.
- **Caminho injetável**: o construtor `HistoricoPartidas(Path)` permite que os testes gravem numa pasta temporária, sem tocar no histórico real.
- O projeto também faz **leitura da internet** com `BufferedReader` + `InputStreamReader` (PokeAPI), em `PokeApiCliente.baixar`.

10. Testes unitários com JUnit 5 NOVO

Teste unitário = código que verifica automaticamente se uma pequena parte do sistema faz o que deveria. Se alguém quebrar uma regra no futuro, o teste falha na hora.

10.1 As peças do JUnit usadas

Recurso	Para que serve
`@Test`	Marca um método como teste.
`@BeforeEach` / `@AfterEach`	Roda antes/depois de **cada** teste (criar jogadores novos; apagar a pasta temporária). Cada teste começa do zero.
`@DisplayName("...")`	Nome legível em português no relatório — é a "documentação" do teste.
`assertEquals(esperado, real)`	Confere um valor (ex.: HP 40 após ataque super efetivo).
`assertTrue` / `assertFalse` / `assertNull`	Confere condições.
`assertSame` / `assertNotSame`	Confere se é (ou não) o **mesmo objeto** na memória (ex.: o clone não é o original).
`assertThrows(Classe, () -> ...)`	Confere que o código **lança** aquela exceção, e devolve a exceção para checar a mensagem.
`assertInstanceOf`, `assertDoesNotThrow`	Confere o tipo do objeto / que não houve exceção.

10.2 Exemplo comentado

```
@Test
@DisplayName("Evoluir para a forma errada mostra de quem ela evolui")
void evolucaoErradaLancaExcecao() {
    // Preparar (Arrange): Squirtle ativo e um Charmeleon na mão
    CartaPokemon base = ativoComEnergia(ash, squirtle());
    int indice = naMao(ash, new CartaPokemon("Charmeleon", "Fogo", 75, 35, "Charmander"));

    // Agir (Act) + Verificar (Assert): tem que lançar a exceção certa...
    JogadaInvalidaException erro = assertThrows(JogadaInvalidaException.class,
            () -> ash.evoluir(indice, base, 2));
    // ...e com a mensagem útil
    assertEquals("Charmeleon não evolui de Squirtle! Ele evolui de Charmander.", erro.getMessage());
}
```

Padrão **AAA** (Arrange, Act, Assert): preparar, executar, verificar. Bom de citar.

10.3 Os 55 testes

Classe de teste	Qtd	O que cobre
`modelo/JogadorTest`	22	Nome vazio; Básico como Ativo; evolução não entra direto; não dois Ativos; Banco ≤ 5; posição inválida; 1 energia/rodada; limite de energia; não ataca no Turno 1; não ataca sem energia; super efetivo dobra dano; 1 ataque/rodada; nocaute completo (zona morta, prêmio, banco sobe); evolução mantém energia e dano; evolução errada; restrições de turno na evolução; encontrar evoluções na mão; recuo custa energia; recuo inválido não gasta energia; baralho vazio; **Observer**: ordem dos eventos, sem evento em jogada inválida, ouvinte removido.
`modelo/CartaPokemonTest`	4	Dano não negativo/nocaute, cura limitada, limite de energias 6–8, construtor recusa valores inválidos.
`modelo/TabelaDeTiposTest`	3	Triângulo de fraquezas, sem vantagem ao contrário, maiúsculas/minúsculas.

`efeitos/EfeitosTreinadorTest`	10	Registro (Strategy), Poção cura 30 e respeita o máximo, Poção sem alvo, Cura Total, Troca sem custo, Troca com Banco vazio, Evolução Rápida no Turno 1, Troféu sem efeito, carta que não é de Treinador.
`fabrica/FabricaEBaralhosTest`	4	Valores padrão da PokeAPI, clone independente, os 3 times têm 40 cartas e toda evolução tem a pré-evolução no time, treinadores e prêmios.
`persistencia/HistoricoPartidasTest`	7	Arquivo inexistente, grava e lê igual, cabeçalho único, linha corrompida, `;` no nome, placar ordenado, erro de disco vira `PersistenciaException`.
`servicos/PokeApiClienteTest`	5	Lê HP/ataque, lê o nome certo, traduz tipo, formata nome, recusa número inválido — tudo **sem internet**, com JSON de exemplo.

10.4 Como rodar

Dois cliques em `rodar-testes.bat`. Ele compila código + testes com o JUnit no classpath e executa `java -jar lib\junit-platform-console-standalone-1.11.4.jar execute --class-path bin --scan-class-path`, que procura sozinho todas as classes terminadas em `Test`. O resultado esperado é **55 tests successful, 0 failed**.

> **Dica para a demonstração:** mostrar os testes rodando é rápido e impressiona. Se der tempo, altere `EfeitoPocao.CURA` para 40, rode os testes, mostre o teste "Poção cura 30" falhando, e volte para 30. Isso demonstra *para que servem* os testes.

11. Referência: todas as classes e métodos

Use como consulta. Coluna "Quem chama" ajuda a responder "de onde vem isso?" na banca.

11.1 Pacote `modelo`

`Carta` (abstrata)

Base de todas as cartas. Atributos privados `nome` e `tipoCarta` ("Pokemon", "Energia", "Treinador").

Método	O que faz
`Carta(nome, tipoCarta)`	Construtor (chamado pelas filhas com `super(...)`).
`getNome()`, `getTipoCarta()`	Getters.
`toString()`	"Nome (Tipo)"; sobrescrito nas filhas.

`CartaPokemon extends Carta`

Atributos: `tipoElemento`, `hpMaximo`, `hpAtual`, `danoAtaque`, `energiasAnexadas` (List), `evoluiDe` (null = básico), `limiteEnergias` (sorteado 6–8), `evoluiuNesteTurno`, `numeroDex` (para a imagem).

Método	O que faz	Quem chama
`CartaPokemon(nome, tipo, hp, dano)` e `(…, evoluiDe)`	Construtores (sobrecarga). Lançam `IllegalArgumentException` se HP ≤ 0 ou dano < 0 **NOVO**.	`CartaPokemonFactory`, testes
`receberDano(dano)`	Diminui o HP sem deixar negativo.	`Jogador.atacar`, `executarEvolucao`
`curar(qtd)`	Aumenta o HP sem passar do máximo.	`EfeitoPocao`, `EfeitoCuraTotal`
`anexarEnergia(energia)`	Adiciona energia se não estiver no limite; devolve `boolean`.	`Jogador.anexarEnergia`, `executarEvolucao`
`isBasico()`	`evoluiDe == null`.	`Jogador`, tela, Bot
`isNocauteado()`	`hpAtual <= 0`.	`Jogador.atacar`
`getQuantidadeEnergias()`, getters/setters	Consulta de estado; `setEvoluiuNesteTurno`, `setNumeroDex`.	Vários

`CartaEnergia extends Carta` e `CartaTreinador extends Carta`

Método	O que faz
`CartaEnergia(nome, tipoElemento)`	Energia de um tipo. Criada na hora por `Jogador.anexarEnergia` (energia "ilimitada").
`CartaTreinador(nome, efeito)`	Guarda o nome do efeito e já resolve a **estratégia** via `EfeitoTreinadorRegistro.obter(efeito)`.
`getEstrategia()`	Devolve a `EfeitoTreinador` (Strategy) desta carta.
`getEfeito()`, `toString()`	Nome do efeito; descrição textual.

`Jogador` — o coração das regras

Atributos: `nome`; listas `baralho`, `mao`, `banco`, `zonaMorta`, `premios`, `ouvintes`; `pokemonAtivo`; flags `energiaAnexadaNestaRodada`, `atacouNestaRodada`, `recuouNesteTurno`. Constantes `LIMITE_BANCO`=5, `CUSTO_RECUO`=1, `TURNO_MINIMO_ATAQUE_E_EVOLUCAO`=2.

Método	O que faz (e o que lança)	Quem chama
`Jogador(nome)`	Nome vazio → `IllegalArgumentException`.	Tela, testes
`adicionarOuvinte` / `removerOuvinte` **NOVO**	Inscreve/cancela um `OuvinteJogo` (Observer).	`AppGUI.iniciarPartida`, `App`, testes

`notificar(tipo, msg)` **NOVO**	Cria um `EventoJogo` e avisa todos os ouvintes.	Os próprios métodos do Jogador e as estratégias
`adicionarAoBaralho`, `embaralhar`	Monta e embaralha (`Collections.shuffle`).	`BaralhosPadrao`, tela
`comprarCarta()`	Tira o topo do baralho para a mão. Vazio → `BaralhoVazioException`. Evento `COMPRA`.	`iniciarPartida`, `iniciarTurno`
`colocarPokemonEmCampo(i, destino)`	0 = Ativo, 1 = Banco. Valida: é Pokémon, é Básico, Ativo livre, Banco < 5.	`acaoClicarCartaPokemon`, Bot
`anexarEnergia(pokemon)`	1 por rodada; respeita o limite da carta. Evento `ENERGIA`.	`abrirMenuPokemon`, Bot
`getPokemonDoCampoPorIndice(i)`	0 = Ativo; 1–5 = Banco. Devolve `null` se não existir.	Terminal
`evoluir(i, alvo, turno)`	Confere Turno ≥ 2 e "1 evolução por turno", depois chama `executarEvolucao`.	`evoluirPokemonEscolhido`, Bot
`executarEvolucao(i, alvo)`	Troca a carta pela evolução, levando energias e dano. Valida nome da pré-evolução. Público porque a `EfeitoEvolucaoRapida` pula as regras de turno.	`evoluir`, `EfeitoEvolucaoRapida`
`usarCartaTreinador(i, params)`	Valida a carta e delega para `estrategia.aplicar(this, i, params)` (Strategy).	`acaoClicarCartaTreinador`, `usarEvolucaoRapidaFluxo`, terminal
`atacar(oponente, turno)`	Valida turno, 1 ataque, Ativos e energia; calcula dano (×2 se `TabelaDeTipos.eSuperEfetivo`); se nocautear, `resolverNocaute`. Eventos `ATAQUE`, `NOCAUTE`, `PREMIO`, `TROCA_DE_ATIVO`.	`acaoAtacar`, Bot
`resolverNocaute` (privado)	Zona morta, prêmio para a mão do atacante, sobe o 1º do Banco.	`atacar`
`trocarAtivoComBanco(iBanco)`	Troca sem custo. Valida Ativo, Banco vazio e posição.	`recuar`, `EfeitoTroca`
`recuar(iBanco)`	1 vez por turno, custa 1 energia — **só paga depois** que a troca deu certo.	`abrirMenuPokemon`
`encerrarRodada()`	Zera as flags da rodada. Evento `FIM_DE_TURNO`.	Tela ao passar/atacar, Bot
`resetarEvolucoesDoTurno()`	Libera todos para evoluir de novo.	`iniciarTurno`
`indicesDeEvolucoesNaMao(alvo)` **NOVO**	Posições na mão das evoluções do alvo.	`escolherEvolucaoNaMao`, Bot
`indiceDoPrimeiroBasicoNaMao()` **NOVO**	Posição do 1º Básico, ou −1.	Bot
`baseEmCampo(nome)`, `semPokemonEmCampo()`	Consultas (esconder evoluções sem base; checar vitória).	Tela
`cartaDaMao(i)` (privado) **NOVO**	Valida o índice uma vez só (antes repetido em cada ação).	Ações do Jogador
Getters	`getMao`, `getBanco`, `getPremios`, `getPokemonAtivo`, `getTamanhoBaralho`, `is...NestaRodada` ...	Tela, testes

`TabelaDeTipos` **NOVO**

`eSuperEfetivo(tipoAtacante, tipoDefensor)`: consulta um `Map` "tipo → tipo que ele vence" (água→fogo, fogo→planta, planta→água), ignorando maiúsculas. Construtor privado: só métodos estáticos.

11.2 Pacote `efeitos` (Strategy)

Classe	Papel e métodos
`EfeitoTreinador` (interface)	`getNome()`, `getTipoAlvoNecessario()`, `getTextoSelecaoAlvo()` (método `default`), `aplicar(jogador, indiceNaMao, parametros)` — lança `JogadaInvalidaException` se não puder aplicar (a carta fica na mão).
`EfeitoPocao`	Alvo: Pokémon em campo. Cura `CURA` = 30.
`EfeitoCuraTotal`	Alvo: Pokémon em campo. Cura até o máximo.
`EfeitoTroca`	Alvo: posição do Banco. Chama `trocarAtivoComBanco` (recuo grátis, não conta como o recuo do turno).
`EfeitoEvolucaoRapida`	Alvo: carta de evolução + Pokémon. Chama `executarEvolucao` direto (ignora turno). Guarda a referência da própria carta, porque o índice muda quando a evolução sai da mão.
`EfeitoNenhum`	Null Object (Troféu): `aplicar` lança "O Troféu é só uma carta-prêmio".
`EfeitoTreinadorRegistro`	`Map` estático nome→estratégia preenchido num bloco `static`. `obter(nome)` devolve a estratégia ou `EfeitoNenhum`.
`TipoAlvo` (enum)	`POKEMON_EM_CAMPO`, `POKEMON_NO_BANCO`, `CARTA_EVOLUCAO_E_ALVO`, `NENHUM` — diz à tela qual diálogo abrir.
`ParametrosEfeito`	Os alvos escolhidos: `alvo`, `indiceBanco`, `indiceCartaEvolucao`. Setters devolvem `this` (encadeamento: `new ParametrosEfeito().setAlvo(p).setIndiceCartaEvolucao(2)`).

11.3 Pacote `eventos` (Observer) NOVO

Classe	Papel
`OuvinteJogo`	Interface funcional: `aoOcorrerEvento(EventoJogo)`.
`EventoJogo`	`record` com `tipo()` e `mensagem()`.
`TipoEvento`	COMPRA, POKEMON_EM_CAMPO, ENERGIA, EVOLUCAO, TREINADOR, ATAQUE, NOCAUTE, PREMIO, TROCA_DE_ATIVO, FIM_DE_TURNO, INFORMACAO.
`OuvinteConsole`	Imprime a mensagem com `System.out.println` (usado pelo terminal).

11.4 Pacote `excecoes` NOVO

Ver seção 7. Todas têm construtor com mensagem; `PokeApiException` e `PersistenciaException` também aceitam a **causa**; `BaralhoVazioException` monta a mensagem sozinha e guarda `getNomeJogador()`.

11.5 Pacote `fabrica`

Método	O que faz
`CartaPokemonFactory.criarDaPokeAPI(dex, nome, tipo, hp, dano, evoluiDe)`	Aplica HP 50 / dano 20 se a API não informar; registra o número da Pokédex.
`criarFixo(...)` (2 versões)	Cria carta com valores conhecidos (times prontos).
`clonar(modelo)`	Cópia nova e independente (HP cheio, sem energia). Usada para reiniciar partida com a mesma equipe.
`BaralhosPadrao.adicionarCartasTreinador(j)` NOVO	15 cartas: 4 Poção, 3 Troca, 4 Evolução Rápida, 4 Cura Total.
`BaralhosPadrao.adicionarPremios(j)` NOVO	2 Troféus na pilha de prêmios (`QUANTIDADE_PREMIOS`).
`adicionarTimeAgua/Fogo/Planta`	40 Pokémon de cada time (via `adicionarCopias`).

`TimeFixo` (enum) NOVO	AGUA, FOGO, PLANTA com `getNome()`, `getEmoji()` e `montar(jogador)` — cada constante guarda uma referência de método (`Consumer<Jogador>`) para o montador do seu time.

11.6 Pacotes `persistencia` e `servicos` NOVO

Método	O que faz
`HistoricoPartidas()` / `(Path)`	Arquivo padrão `dados/historico_partidas.csv` ou outro (testes).
`registrar(resultado)`	Acrescenta uma linha (seção 9).
`carregar()`	Lê todas as partidas; pula linhas ruins.
`vitoriasPorJogador()`	`Map` nome → vitórias, ordenado.
`ResultadoPartida`	Record: dataHora, vencedor, perdedor, modo, turnos, motivo.
`PokeApiCliente.buscarNomes(offset, limite)`	Nomes de uma geração (lista do montador de time).
`buscarPokemon(dex)`	Baixa `/pokemon/{n}` e `/pokemon-species/{n}`, extrai nome, tipo, HP, ataque e pré-evolução, cria a carta pela fábrica.
`buscarLinhaEvolutiva(dex)`	Volta pela pré-evolução até o básico (Charizard → Charmeleon → Charmander).
`buscarTipo(dex)`	Só o tipo (ícone da lista).
`traduzirTipo`, `capitalizar`	"fire"→"Fogo"; "mr-mime"→"Mr mime".
`baixar(url)` (privado)	GET com tempo limite de 8 s; erro HTTP ou `IOException` → `PokeApiException`; `finally` desconecta.
`extrairStat`, `extrairTipo`, `extrairNome`	Regex sobre o JSON (sem biblioteca externa). Visíveis no pacote para os testes.

11.7 Pacote `ui`

`AppGUI` — métodos agrupados por responsabilidade

Grupo	Métodos
Início e partidas	`main`, `iniciar`, `montarComposicao` NOVO, `capturarComposicao`, `construirJogadorFresco`, `iniciarPartida`, `reiniciarComMesmaEquipe`, `reiniciarTrocandoEquipe`
Menus e escolha de time	`escolherModoDialog`, `pedirNomeEstilizado`, `escolherTimeDialog` (usa `TimeFixo` + `EnumSet`), `abrirConstrutorDeTime` (time customizado pela PokeAPI), `construirTimeAleatorioBot(ComLoading)`
Eventos e exceções NOVO	`aoEventoDoJogo` (ouvinte), `registrarMensagem`, `executarJogada`, `tentarJogadaDoBot`
Ciclo de turno	`iniciarTurno`, `trocarTurno`, `executarTurnoBot`, `verificarVitoriaAposAtaque` NOVO, `encerrarJogoComDerrota`, `salvarResultadoEMontarPlacar` NOVO, `mostrarTelaFimDeJogo`
Ações do jogador	`acaoClicarCartaPokemon`, `abrirMenuPokemon` (energia/evoluir/recuar), `evoluirPokemonEscolhido`, `escolherEvolucaoNaMao` NOVO, `acaoClicarCartaTreinador` (switch no `TipoAlvo`), `usarEvolucaoRapidaFluxo`, `escolherPokemonEmCampoDialog`, `acaoAtacar`, `acaoPassarTurno`
Desenho do tabuleiro	`montarJanela`, `atualizarTela`, `montarCampoJogador`, `montarMaoJogador`, `mostrarZonaMortaDialog`, `abrirHistoricoDialog`, `mostrarFlashDeAtaque`, `mostrarNotificacao` / `criarToast` / `reposicionarToasts`

Imagens	`baixarImagemBuffered`, `baixarSprite` **NOVO**, `carregarSprite`, `obterSpriteParaLista`, `obterTipoParaLista`, desenhos do menu (pokébola, desfoque)

`App`, `CartaVisual`, `SorteioMoeda`

Classe	Resumo
`App`	Versão em terminal com `Scanner`. Usa o mesmo `Jogador`, `TimeFixo`, `OuvinteConsole` e trata `JogadaInvalidaException`, `BaralhoVazioException` e `InputMismatchException`. Usa a carta de Treinador de forma genérica pelo `TipoAlvo` (sem comparar nomes).
`CartaVisual extends JButton`	Desenha a carta sobrescrevendo `paintComponent` (Graphics2D): cores pelo tipo, barra de HP, imagem, energias. Efeito de "levantar" ao passar o mouse (`MouseAdapter`).
`SorteioMoeda extends JDialog`	Diálogo modal de Cara ou Coroa com animação por `javax.swing.Timer`. `sortearQuemComeca(n1, n2)` devolve `true` se o jogador 1 começa.

12. Roteiro da apresentação (10–15 minutos)

A Etapa 6 exige: slides e/ou demo ao vivo, demonstração do sistema, explicação das decisões e padrões, **participação de todos** e respostas à banca. Sugestão de divisão — vocês podem trocar os blocos, mas **os dois devem conseguir explicar todos**.

Tempo	Quem	Bloco	O que falar / mostrar
0:00–1:30	Renato	Abertura	O que é o sistema (TCG em Java + Swing, PvP e Solo, PokeAPI). Tecnologias: Java 17, Swing, JUnit 5, sem bibliotecas externas no jogo.
1:30–3:00	Lucas	Arquitetura	Slide dos 8 pacotes (diagrama da seção 3). Regra: o `modelo` não conhece tela, arquivo nem internet — por isso dá para ter duas interfaces e testar sem janela.
3:00–7:00	Renato (joga) + Lucas (narra)	**Demo ao vivo**	(1) Abrir com `rodar.bat`; (2) escolher modo Solo e time Fogo; (3) Cara ou Coroa; (4) clicar "Atacar" no Turno 1 → mostrar a notificação de **exceção**; (5) baixar Pokémon, anexar energia, tentar 2ª energia → outra exceção; (6) passar turno, ver o Bot jogar (eventos do **Observer** aparecendo); (7) usar uma Poção (**Strategy**); (8) atacar com super efetividade; (9) abrir "Ver Histórico".
7:00–9:00	Lucas	Padrões de projeto	Strategy: problema (if por nome de carta), solução (interface + classes), ganho (carta nova sem mexer em nada). Observer: problema (`System.setOut`, acentos quebrados), solução (ouvintes), ganho (tela, console e testes recebem os mesmos eventos). Mencionar que a fábrica é Simple Factory (não-GoF).
9:00–10:30	Renato	Exceções e arquivos	Hierarquia; verificada × não verificada (e por quê); `executarJogada` como ponto único. Mostrar `dados/historico_partidas.csv` aberto no Bloco de Notas e o placar na tela de fim de jogo.
10:30–12:00	Lucas	Testes	Rodar `rodar-testes.bat` ao vivo: 55 testes verdes. Mostrar um teste com `assertThrows`. (Opcional: quebrar a Poção para mostrar um teste falhando.)
12:00–13:00	Renato	Fechamento	Conceitos de POO usados (herança, polimorfismo, encapsulamento, interfaces, enum/record), limitações e próximos passos. Abrir para perguntas.

Sugestão de slides (8 a 10)

1. Capa: nome do projeto, integrantes, disciplina.
2. O sistema: o que faz + print do tabuleiro.
3. Arquitetura: diagrama dos pacotes.
4. Fluxo de uma jogada (diagrama da seção 4.2).
5. Strategy: diagrama + "antes/depois" em 2 linhas.
6. Observer: diagrama + "antes/depois".
7. Exceções: hierarquia + exemplo de mensagem.
8. Coleções e arquivo: tabela curta + trecho do CSV.
9. Testes: print dos 55 testes passando.
10. Conclusão: o que aprendemos / próximos passos.

> **Plano B para a demo:** o modo Solo e o time customizado precisam de internet (PokeAPI). Se a internet da sala falhar, use **PvP com os times fixos** (funciona offline, só sem as imagens). Tenha também prints/GIF da demo nos slides. Teste o projeto no computador da apresentação antes.

Limitações (diga antes que perguntem — mostra maturidade)

- A leitura do JSON da PokeAPI é feita com regex, não com uma biblioteca de JSON (decisão: não usar dependências externas). É mais frágil se a API mudar o formato — por isso há testes com JSON de exemplo.
- O `AppGUI` ainda é grande (~1900 linhas): concentra telas e a IA do Bot. Próximo passo seria separar o Bot numa classe própria (poderia até ser outra Strategy: `EstrategiaDoBot`).
- A IA do Bot é simples (sempre a mesma sequência de ações).
- Não há "salvar partida no meio" — o arquivo guarda só resultados.

13. Perguntas prováveis da banca (com respostas)

Padrões de projeto

1. Quais padrões GoF vocês aplicaram?

Strategy, nos efeitos das cartas de Treinador, e Observer, nos eventos da partida. Também usamos uma Simple Factory para criar cartas e um Null Object (`EfeitoNenhum`), mas esses não contamos como GoF.

2. Qual problema o Strategy resolve aqui? Por que não um switch?

Cada carta de Treinador tem um comportamento diferente para a mesma ação "usar". Com switch/if, cada carta nova obrigaria a mexer no Jogador e na tela (e antes era exatamente assim, comparando o nome). Com Strategy, cada efeito é uma classe que implementa `EfeitoTreinador`; o Jogador só chama `aplicar`. Carta nova = classe nova + uma linha no registro. É o princípio aberto/fechado.

3. Quem é o contexto, a estratégia e as estratégias concretas?

Contexto: `Jogador.usarCartaTreinador` (e a `CartaTreinador`, que guarda a estratégia). Estratégia: a interface `EfeitoTreinador`. Concretas: `EfeitoPocao`, `EfeitoCuraTotal`, `EfeitoTroca`, `EfeitoEvolucaoRapida`, `EfeitoNenhum`.

4. Como a carta sabe qual estratégia usar?

No construtor da `CartaTreinador`, ela pede ao `EfeitoTreinadorRegistro.obter(nomeDoEfeito)`, que procura num `HashMap`. Se o nome não existir, devolve o `EfeitoNenhum` (nunca `null`).

5. Explique o Observer no projeto.

O `Jogador` é o Subject: tem uma lista de `OuvinteJogo` e, a cada ação (ataque, evolução, nocaute...), chama `notificar`, que entrega um `EventoJogo` a todos. Os observadores são a tela (mostra notificação e histórico), o `OuvinteConsole` do terminal e as listas dos testes. O Jogador não sabe quem está ouvindo.

6. Por que o Observer é melhor que o System.out que vocês tinham?

Três motivos: (1) as regras não dependem mais do console; (2) dá para testar que um evento aconteceu; (3) acabou o bug de acentos — a saída falsa convertia cada byte UTF-8 numa letra. E cada ouvinte pode reagir diferente: a tela esconde a carta que o Bot comprou usando o tipo `COMPRA`.

7. A CartaPokemonFactory não é um Factory Method?

Não. Factory Method (GoF) usa herança: uma classe criadora abstrata com um método de criação que as subclasses sobrescrevem. A nossa é uma Simple Factory: métodos estáticos que centralizam a criação. Ela resolveu duplicação real (4 lugares criando cartas com a mesma lógica), mas não contamos como padrão GoF.

8. Que outro padrão caberia no projeto?

Prototype (o `clonar` poderia virar um método `copiar()` na própria carta), State (estados do turno), ou um Strategy para a IA do Bot (níveis de dificuldade).

Exceções

9. Por que criar exceções próprias em vez de usar Exception?

O tipo da exceção diz *o que* aconteceu, e cada tipo é tratado de um jeito: jogada inválida mostra aviso; baralho vazio encerra a partida; falha de rede deixa o Bot pular o Pokémon. Com `Exception` genérica não daria para distinguir.

10. Diferença entre exceção verificada e não verificada? Por que essa escolha?

Verificada (herda de `Exception`) o compilador obriga a tratar ou declarar; não verificada (`RuntimeException`) não. Regras do jogo são não verificadas: podem acontecer em qualquer ação e tratamos tudo num ponto só (`executarJogada`). Rede e disco são verificadas: são falhas externas previsíveis e queremos que o compilador obrigue quem chama a decidir o que fazer.

11. Onde as exceções são tratadas?

Jogada do humano: `AppGUI.executarJogada`. Jogada do Bot: `tentarJogadaDoBot`. Baralho vazio: `iniciarTurno`. PokeAPI: no montador de time e no sorteio do Bot. Histórico: `salvarResultadoEMontarPlacar`. Terminal: no loop do `App.main`.

12. O que acontece com o jogo quando uma jogada é inválida?

Nada muda: as validações acontecem **antes** de alterar qualquer coisa, então a exceção sai com o estado intacto, e nenhum evento é notificado. Temos testes que conferem isso (ex.: recuo inválido não gasta energia; Poção sem alvo continua na mão).

13. Para que serve passar a "causa" na exceção?

`new PokeApiException("Verifique sua internet", e)` mostra uma mensagem amigável ao usuário, mas guarda a `IOException` original em `getCause()` para quem for depurar.

Coleções e arquivos

14. Por que List no baralho e na mão, e não Set?

Porque a ordem importa (topo do baralho, posição na mão) e há cartas repetidas (4 Squirtles). Set não guarda ordem nem aceita repetidos.

15. Onde usam Set e Map? Por quê?

Set: times disponíveis (`EnumSet`, cada time uma vez só), números já visitados na linha evolutiva (evita laço), cache de downloads em andamento. Map: registro de estratégias (nome → efeito), tabela de tipos, tradução de tipos, placar (nome → vitórias, `LinkedHashMap` para manter a ordem do ranking), cache de imagens.

16. Como funciona a leitura/escrita do arquivo?

`HistoricoPartidas` grava uma linha CSV por partida com `Files.newBufferedWriter` em modo APPEND e UTF-8, criando a pasta e o cabeçalho na 1ª vez. Lê com `BufferedReader`, transforma cada linha num `ResultadoPartida` e pula linhas corrompidas. Tudo com try-with-resources, e erros viram `PersistenciaException`.

17. O que é try-with-resources?

`try (BufferedReader r = ...) { }`: o recurso declarado nos parênteses é fechado automaticamente no fim, mesmo se der exceção. Evita esquecer o `close()` e deixar o arquivo travado.

18. E se alguém apagar ou estragar o arquivo?

Se não existir, o histórico começa vazio. Linhas inválidas são puladas e contadas. Se o disco falhar, aparece um aviso na tela de fim de jogo e o jogo continua.

Testes

19. O que vocês testaram e por quê?

As regras de negócio, que são o mais importante e mais fácil de quebrar: `Jogador` (22 testes), cartas, tabela de tipos, cada estratégia, fábrica e times, histórico em arquivo e a leitura da PokeAPI. A interface gráfica não tem teste unitário — foi testada manualmente.

20. Como testar o arquivo sem bagunçar o histórico real?

O `HistoricoPartidas` aceita o caminho no construtor. O teste cria uma pasta temporária no `@BeforeEach` e apaga no `@AfterEach`.

21. Como testar a PokeAPI sem internet?

Separamos baixar (rede) de interpretar (regex). Os testes chamam os métodos de interpretação com um JSON de exemplo, no mesmo formato da API.

22. Como se testa que uma exceção é lançada?

`assertThrows(JogadaInvalidaException.class, () -> jogador.atacar(oponente, 1))`. Ele devolve a exceção, e dá para conferir a mensagem com `assertEquals`.

23. Algum teste pegou bug de verdade?

Sim: o teste da Poção revelou que ela curava 40 e não 30 como o texto da carta dizia.

POO

24. Onde há herança? E polimorfismo?

Herança: `CartaPokemon/CartaEnergia/CartaTreinador extends Carta`; exceções; `CartaVisual extends JButton`. Polimorfismo: `estrategia.aplicar(...)`, `ouvinte.aoOcorrerEvento(...)`, `toString()`, `paintComponent`.

25. Por que Carta é abstrata?

Não existe "carta genérica" no jogo: toda carta é Pokémon, Energia ou Treinador. Abstrata impede `new Carta(...)` e concentra o que é comum (nome, tipo).

26. Mostre encapsulamento.

`hpAtual` é privado; só muda por `receberDano` (nunca negativo) e `curar` (nunca acima do máximo). Ninguém consegue deixar um Pokémon com HP −10.

27. Interface ou classe abstrata — qual a diferença e quando usaram cada uma?

Classe abstrata compartilha estado e código (Carta tem nome e tipo). Interface é só contrato e uma classe pode implementar várias. `EfeitoTreinador` e `OuvinteJogo` são interfaces porque não há estado comum — só o comportamento muda.

28. O que é um record e por que usaram?

Classe imutável de dados, com construtor, getters, equals/hashCode/toString gerados (Java 16+). `EventoJogo` e `ResultadoPartida` são só dados que não devem mudar depois de criados.

29. Para que servem os enums TipoAlvo, TipoEvento e TimeFixo?

Substituem strings/números mágicos por valores fixos verificados pelo compilador. `TimeFixo` ainda tem comportamento (`montar`), eliminando cadeias de `if`.

Código e decisões

30. Como vocês evitaram duplicação?

Validação da mão num método (`cartaDaMao`); troca de Ativo usada pelo recuo e pela carta Troca; escolha de evolução e checagem de vitória extraídas na tela; `montarComposicao` usado ao iniciar e ao trocar de time; atalhos `usarPocao/usarTroca` removidos; times num enum.

31. Por que a tela usa `this::aoEventoDoJogo`?

É uma referência de método: como `OuvinteJogo` tem um único método, qualquer método com a mesma assinatura serve como ouvinte. Evita criar uma classe só para isso.

32. Por que o Bot usa threads?

Para sortear o time, ele busca vários Pokémon na PokeAPI **ao mesmo tempo** (`ExecutorService`), juntando os resultados numa `BlockingQueue`. A tela mostra "carregando" sem travar.

33. Como funciona a super efetividade?

`TabelaDeTipos` tem um `Map` "tipo → tipo que ele vence". No ataque, se `eSuperEfetivo(atacante, defensor)`, o dano dobra.

34. Como a evolução mantém o dano?

`executarEvolucao` calcula `hpMaximo − hpAtual` da forma anterior, passa as energias para a evolução, aplica o mesmo dano nela e a coloca no lugar (Ativo ou mesma posição do Banco).

35. Se eu quiser uma carta nova "Super Poção (+60)", o que muda?

Criar `EfeitoSuperPocao implements EfeitoTreinador` (cópia da Poção com 60), registrar no `EfeitoTreinadorRegistro` e adicionar ao baralho em `BaralhosPadrao`. Nada no `Jogador` nem no `AppGUI` — a tela descobre o diálogo pelo `TipoAlvo`.

14. Checklist do dia da defesa

- ☐ `git pull` no computador da apresentação: está na versão mais nova do GitHub?
- ☐ JDK 17+ instalado lá (`java -version`).
- ☐ `rodar.bat` abre o jogo; `rodar-testes.bat` mostra 55 testes passando.
- ☐ Internet funcionando? Se não, plano B: PvP com times fixos.
- ☐ Apagar `dados/historico_partidas.csv` antes, se quiser começar o placar do zero na demo (ou jogar 1–2 partidas antes para ter placar).
- ☐ Slides prontos, com prints de reserva.
- ☐ Ensaio cronometrado: 10–15 minutos.
- ☐ Cada um explicou para o outro, em voz alta: Strategy, Observer, exceções (verificada × não), arquivo, coleções, testes.
- ☐ Cada um sabe abrir e explicar: `Jogador.atacar`, `Jogador.notificar`, `AppGUI.executarJogada`, `EfeitoPocao.aplicar`, `HistoricoPartidas.registrar` e um teste com `assertThrows`.

> **Frase-resumo para fechar:** "O sistema separa regras, interface e acesso externo em pacotes; usa Strategy para que cada carta tenha seu comportamento sem `if`s, Observer para que as regras avisem a tela sem depender dela, exceções específicas com mensagens úteis, coleções escolhidas pela necessidade, histórico em arquivo e 55 testes automatizados garantindo as regras."